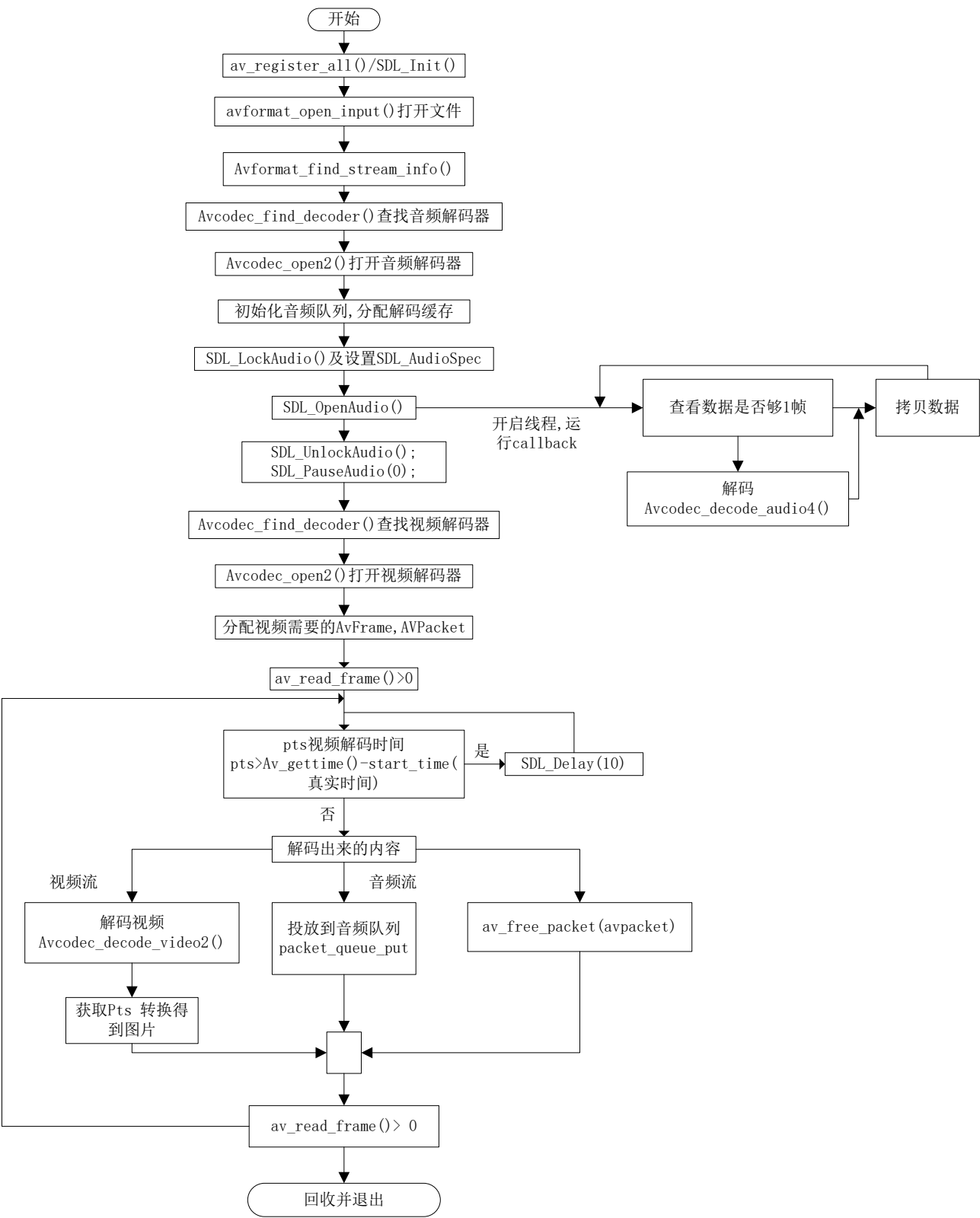


四.视音频播放混合

流程



在原有的播放器 demo 中扩展音频解码播放

首先将头和库文件扩充一下

```
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include\  
             $$PWD/SDL2-2.0.10/include  
  
LIBS += $$PWD/ffmpeg-4.2.2/lib/avcodec.lib\  
        $$PWD/ffmpeg-4.2.2/lib/avdevice.lib\  
        $$PWD/ffmpeg-4.2.2/lib/avfilter.lib\  
        $$PWD/ffmpeg-4.2.2/lib/avformat.lib\  
        $$PWD/ffmpeg-4.2.2/lib/avutil.lib\  
        $$PWD/ffmpeg-4.2.2/lib/postproc.lib\  
        $$PWD/ffmpeg-4.2.2/lib/swresample.lib\  
        $$PWD/ffmpeg-4.2.2/lib/swscale.lib\  
        $$PWD/SDL2-2.0.10/lib/x86/SDL2.lib
```

```
extern "C"  
{  
    #include "libavcodec/avcodec.h"  
    #include "libavformat/avformat.h"  
    #include "libavutil/pixfmt.h"  
    #include "libswscale/swscale.h"  
    #include "libavformat/avformat.h"  
    #include "libavdevice/avdevice.h"  
    #include "libswresample/swresample.h"  
    #include "libavutil/time.h"  
    #include "SDL.h"  
}
```

main 函数修改

```
#undef main  
int main(...)  
{  
    ...  
}
```

我们在原有的视频项目的基础上，添加音频部分

1. 添加音频需要的宏和全局变量

```
#define AVCODEC_MAX_AUDIO_FRAME_SIZE    192000 //1 second of 48khz 32bit audio  
#define SDL_AUDIO_BUFFER_SIZE    1024    //
```

```
AVFrame        wanted_frame;  
PacketQueue    audio_queue;  
int            quit = 0;
```

2. 添加回调函数，解码函数，查找流函数

//回调函数

```
void audio_callback(void *userdata, Uint8 *stream, int len);
```

//解码函数

```
int audio_decode_frame(AVCodecContext *pcodec_ctx, uint8_t *audio_buf, int buf_size);
```

//找 auto_stream

```
int find_stream_index(AVFormatContext *pformat_ctx, int *video_stream, int *audio_stream);
```

3. 添加音频部分对应的变量

//添加音频需要的变量

```
int audioStream = -1; //音频解码器需要的流的索引
```

```
AVCodecContext *pAudioCodecCtx = NULL; //音频解码器信息指针
```

```
AVCodec *pAudioCodec = NULL; //音频解码器
```

//SDL

```
SDL_AudioSpec wanted_spec; //SDL 音频设置
```

```
SDL_AudioSpec spec; //SDL 音频设置
```

4. 添加 SDL 初始化

在 ffmpeg 初始化之后，添加 SDL 初始化

//SDL 初始化

```
if (SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO | SDL_INIT_TIMER))
{
    qDebug() << "Couldn't init SDL: " << SDL_GetError();
    return;
}
```

5. 直接使用查找视频流函数，给视频音频流索引赋值.

//查找音频视频流索引

```
if (find_stream_index(pFormatCtx, &videoStream, &audioStream) == -1)
{
    qDebug() << "Couldn't find stream index";
    return;
}
```

6. 查找和打开音频对应的解码器

//找到对应的音频解码器

```
pAudioCodecCtx = pFormatCtx->streams[audioStream]->codec;
```

```
pAudioCodec = avcodec_find_decoder(pAudioCodecCtx->codec_id);
```

```
if (!pAudioCodec)
```

```
{
    qDebug() << "Couldn't find decoder";
    return;
}
```

//打卡音频解码器

```
avcodec_open2(pAudioCodecCtx, pAudioCodec, NULL);
```

7. 设置音频信息，用来打开音频设备

//设置音频信息，用来打开音频设备。

```
wanted_spec.freq          = pAudioCodecCtx->sample_rate; //采样频率
wanted_spec.format        = AUDIO_S16SYS; //精度
wanted_spec.channels      = pAudioCodecCtx->channels; //通道数
wanted_spec.silence       = 0; //设置静音值
wanted_spec.samples       = SDL_AUDIO_BUFFER_SIZE; //采样点
wanted_spec.callback      = audio_callback; //回调函数
wanted_spec.userdata      = pAudioCodecCtx; //回调函数参数
```

8. 打开音频设备

//打开音频设备

```
SDL_AudioDeviceID id = SDL_OpenAudioDevice(NULL, 0, &wanted_spec, &spec, 0);
if( id < 0 ) //第二次打开 audio 会返回-1
{
    qDebug() << "Couldn't open Audio: " << SDL_GetError() ;
    return;
}
```

9. 设置参数，供解码时候用

//设置参数，供解码时候用，swr_alloc_set_opts 的 in 部分参数

```
wanted_frame.format        = AV_SAMPLE_FMT_S16;
wanted_frame.sample_rate   = spec.freq;
wanted_frame.channel_layout = av_get_default_channel_layout(spec.channels);
wanted_frame.channels      = spec.channels;
```

10. 初始化队列

//初始化队列

```
packet_queue_init(&audio_queue);
```

11. 播放音频

// SDL 播放声音 0 播放

```
SDL_PauseAudioDevice(id, 0);
```

12. 读音频数据帧

```
if ( packet->stream_index == audioStream)
{
    packet_queue_put(&audio_queue, packet);
}
else
{
    av_free_packet(packet);
}
```

13. 回收数据

```
while( audio_queue.nb_packets != 0)
```

```

{
    SDL_Delay(100);
}
//回收空间
avcodec_close(pAudioCodecCtx);

```

14. 回调函数定义

//回调函数中将从队列中取数据，解码后填充到播放缓冲区.

```

void audio_callback(void *userdata, Uint8 *stream, int len)
{
    AVCodecContext *pcodec_ctx = (AVCodecContext *) userdata;

    int len1, audio_data_size;

    static uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
    static unsigned int audio_buf_size = 0;
    static unsigned int audio_buf_index = 0;

    /* len 是由 SDL 传入的 SDL 缓冲区的大小，如果这个缓冲未满足，我们就一直往里填充数据 */

    /* audio_buf_index 和 audio_buf_size 标示我们自己用来放置解码出来的数据的缓冲区，*/
    /* 这些数据待 copy 到 SDL 缓冲区，当 audio_buf_index >= audio_buf_size 的时候意味着我*/
    /* 们的缓冲为空，没有数据可供 copy，这时候需要调用 audio_decode_frame 来解码出更*/
    /* 多的帧数据 */
    while (len > 0)
    {
        if (audio_buf_index >= audio_buf_size) {
            audio_data_size = audio_decode_frame(pcodec_ctx,
audio_buf, sizeof(audio_buf));
            /* audio_data_size < 0 标示没能解码出数据，我们默认播放静音 */
            if (audio_data_size < 0) {
                /* silence */
                audio_buf_size = 1024;
                /* 清零，静音 */
                memset(audio_buf, 0, audio_buf_size);
            } else {
                audio_buf_size = audio_data_size;
            }
            audio_buf_index = 0;
        }
        /* 查看 stream 可用空间，决定一次 copy 多少数据，剩下的下次继续 copy */
        len1 = audio_buf_size - audio_buf_index;
        if (len1 > len) {
            len1 = len;
        }

        memset( stream , 0 , len1);
    }
}

```

```

//混音函数 sdl 2.0 版本使用该函数 替换 SDL_MixAudio
SDL_MixAudioFormat(stream, (uint8_t *) audio_buf + audio_buf_index,
                    AUDIO_S16SYS, len1, 100);

len -= len1;
stream += len1;
audio_buf_index += len1;
}
}

```

//对于音频来说，一个 packet 里面，可能含有多帧(frame)数据。

//解码音频帧函数

```

int audio_decode_frame(AVCodecContext *pcodec_ctx, uint8_t *audio_buf, int buf_size)
{

    static AVPacket pkt;
    static uint8_t *audio_pkt_data = NULL;
    static int audio_pkt_size = 0;
    int len1, data_size;
    int sampleSize = 0;

    AVCodecContext *aCodecCtx = pcodec_ctx;
    AVFrame *audioFrame = NULL;
    PacketQueue *audioq = &audio_queue;

    static struct SwrContext *swr_ctx = NULL;
    int convert_len;
    int n = 0;

    for(;;)
    {
        if( quit ) return -1;

        if(packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
        {
            return -1;
        }
        audioFrame = av_frame_alloc();
        audio_pkt_data = pkt.data;
        audio_pkt_size = pkt.size;
        while(audio_pkt_size > 0)
        {
            if( quit ) return -1;

            int got_picture;
            memset(audioFrame, 0, sizeof(AVFrame));

```

```

int ret =avcodec_decode_audio4( aCodecCtx, audioFrame, &got_picture, &pkt);
if( ret < 0 ) {
    qDebug()<<"Error in decoding audio frame. ";
    exit(0);
}

```

```

data_size = audioFrame->nb_samples * wanted_frame.channels * 2;

```

```

if( got_picture )
{

```

```

    if (swr_ctx != NULL)
    {
        swr_free(&swr_ctx);
        swr_ctx = NULL;
    }

```

```

    swr_ctx = swr_alloc_set_opts(NULL, wanted_frame.channel_layout,
        (AVSampleFormat)wanted_frame.format,wanted_frame.sample_rate,
        audioFrame->channel_layout, (AVSampleFormat)audioFrame->format,
        audioFrame->sample_rate, 0, NULL);

```

```

//初始化

```

```

if (swr_ctx == NULL || swr_init(swr_ctx) < 0)
{
    qDebug()<<"swr_init error\n";
    break;
}

```

```

convert_len = swr_convert(swr_ctx, &audio_buf,
    AVCODEC_MAX_AUDIO_FRAME_SIZE,
    (const uint8_t **)audioFrame->data,
    audioFrame->nb_samples);

```

```

}

```

```

audio_pkt_size -= ret;

```

```

if (audioFrame->nb_samples <= 0)
{
    continue;
}

```

```

av_free_packet(&pkt);
return data_size ;

```

```

}

```

```

av_free_packet(&pkt);

```

```

}

```

```

}

```

//查找数据流函数

```
int find_stream_index(AVFormatContext *pformat_ctx, int *video_stream, int
*audio_stream)
{
    assert(video_stream != NULL || audio_stream != NULL);

    int i = 0;
    int audio_index = -1;
    int video_index = -1;

    for (i = 0; i < pformat_ctx->nb_streams; i++)
    {
        if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_VIDEO)
        {
            video_index = i;
        }
        if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_AUDIO)
        {
            audio_index = i;
        }
    }

    //注意以下两个判断有可能返回-1.
    if (video_stream == NULL)
    {
        *audio_stream = audio_index;
        return *audio_stream;
    }

    if (audio_stream == NULL)
    {
        *video_stream = video_index;
        return *video_stream;
    }

    *video_stream = video_index;
    *audio_stream = audio_index;

    return 0;
}
```

同时兼容视频音频

现在当前的逻辑是打不开音频解码器或者视频解码器就退出程序，这显然是不科学的，应该是，如果找不到视频的解码器，那么证明该文件可能是纯音频，那么就执行音频响应的代码。那么考虑到上

面的情况,可以利用 videoStream , audioStream 这种视频音频流的下标来完成. 如果没有找到, 值为初始值-1, 那么我们的做法是设置和解析就不执行.

执行如下

1. 首先将后面声明的变量名, 拿到前面声明.

//视频

```
AVCodecContext *pCodecCtx ; //视频的解码器信息指针
```

```
AVCodec *pCodec ; //视频解码器
```

```
AVFrame *pFrame, *pFrameRGB;// 用来存解码后的数据
```

```
AVPacket *packet;//读取解码前的包
```

```
int numBytes;//帧数据大小
```

```
uint8_t * out_buffer;//存储转化为 RGB 格式数据的缓冲区
```

```
struct SwsContext *img_convert_ctx;//YUV 转 RGB 的结构
```

2. 在视频音频的设置部分, 加入

```
if( audioStream != -1)
```

```
{
```

```
}
```

```
if(videoStream != -1)
```

```
{
```

```
}
```

注意事项:

在解析数据包需要对 AV packet 进行初始化, 这个包之前是在视频部分去初始化的.

如果没有视频那么应该在音频部分也有初始化, 所以最后我们会在外面去给 AV Packet 初始化.

//视频部分

```
if( videoStream != -1) {...
```

```
//获取开始时间
```

```
int64_t start_time = av_gettime();
```

```
int64_t pts = 0; //当前视频帧的 pts
```

//音频部分

```
if( audioStream != -1 ) {...
```

```
packet = (AVPacket *) malloc(sizeof(AVPacket)); //分配一个 packet
```

五.Qt 视频播放器之音视频同步(优化版)

前面已经实现了基本的播放功能, 不过还是有要优化的地方. 比如, 我们看流程, 发现解码视频会影响音频的解码, 这是我們不想看到的. 那么要如何优化来避免呢? 很显然, 为了音频不影响视频, 采用缓冲队列的方式, 同样的, 为了视频不影响音频, 也应该采用缓冲队列的方式. 只是, 在 SDL_OpenAudio 的时候, SDL 会为我们开启一个线程, 而在处理视频的时候需要我们手动开启一个线程. 同时由于, 解码视频发生在线程中, 而解码视频所需要的数据结构在 Run()函数里面, 因此需要 VideoState 结构体来完成.

```
class VideoPlayer ;
```

```

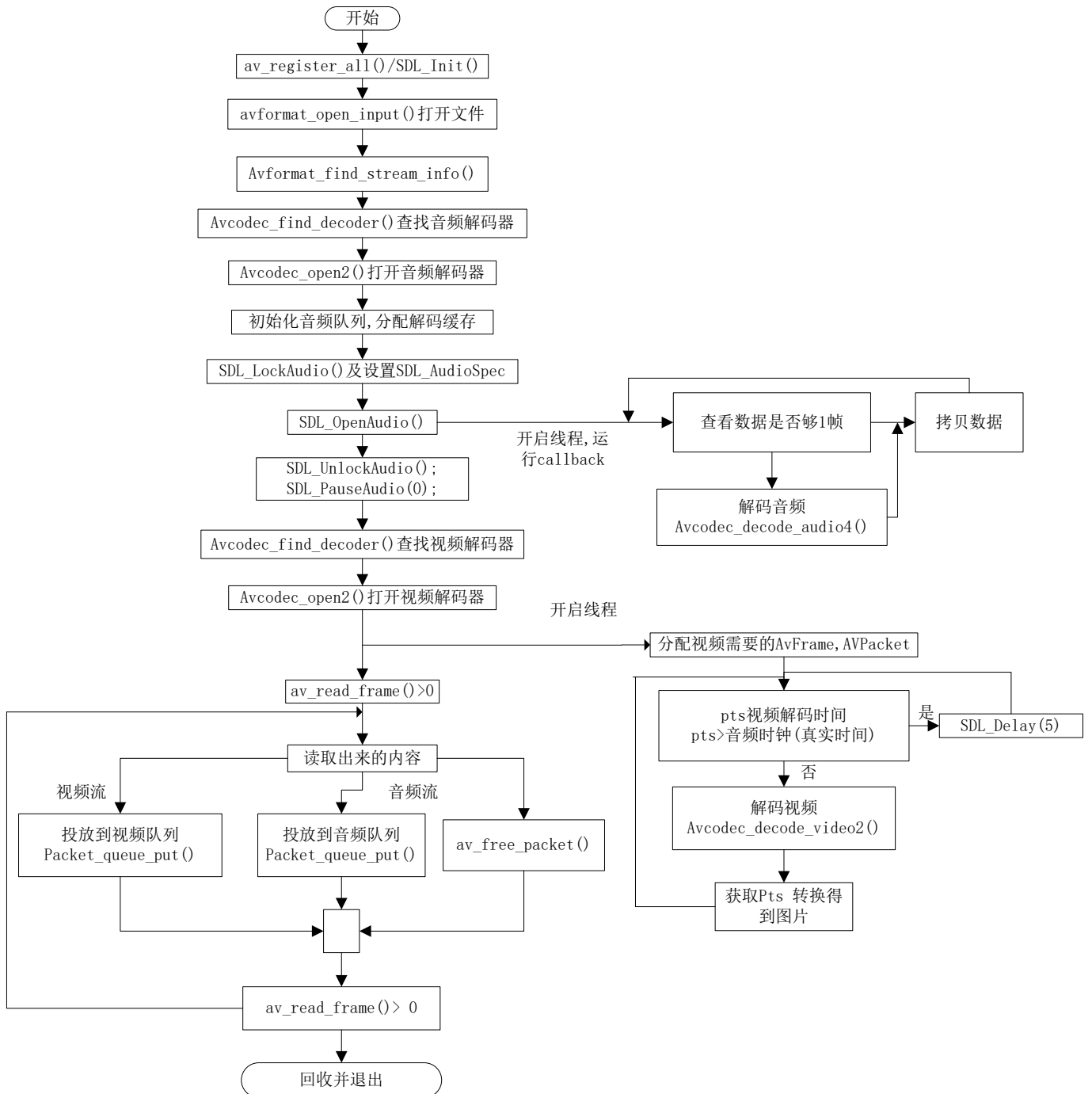
typedef struct VideoState {
    AVFormatContext *pFormatCtx; //相当于视频”文件指针”
    ////////////////音频//////////////////////
    AVStream      *audio_st; //音频流
    PacketQueue    *audioq; //音频缓冲队列
    AVCodecContext *pAudioCodecCtx; //音频解码器信息指针
    int audioStream; //视频音频流索引
    double audio_clock; ///<pts of last decoded frame 音频时钟
    SDL_AudioDeviceID audioID; //音频 ID
    AVFrame out_frame; //设置参数，供音频解码后的 swr_alloc_set_opts 使用。
    /// 音频回调函数使用的量
    uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
    unsigned int audio_buf_size = 0;
    unsigned int audio_buf_index = 0;
    AVFrame *audioFrame;
    ////////////////视频//////////////////////
    AVStream      *video_st; //视频流
    PacketQueue    *videoq; //视频队列
    AVCodecContext *pCodecCtx; //音频解码器信息指针
    int videoStream; //视频音频流索引
    double video_clock; ///<pts of last decoded frame 视频时钟
    SDL_Thread *video_tid; //视频线程 id
    ////////////////

    ////////////////播放控制//////////////////////
    int quit;
    ////////////////
    int64_t start_time; //单位 微秒
    VideoState()
    {
        audio_clock = video_clock = start_time = 0;
    }
    VideoPlayer* m_player; //用于调用函数
} VideoState;

```

VideoState 结构体里面包含了解码音频视频所有的相关信息.

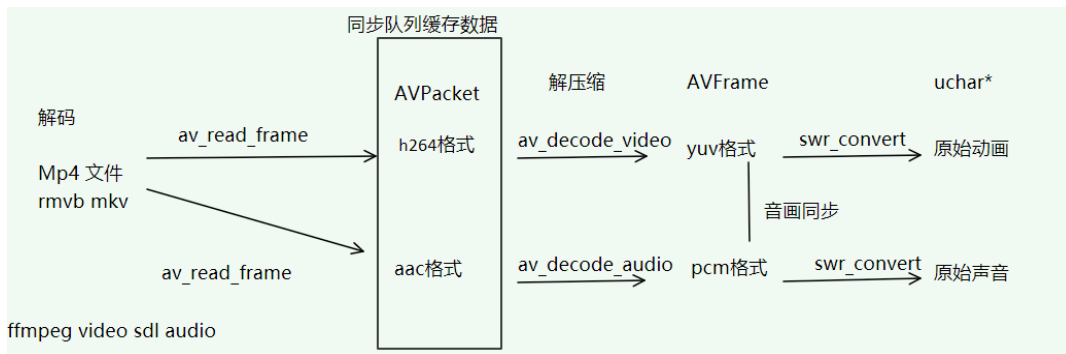
优化的流程图与上一篇文章类似, 如下图



我们可以看出, 优化的程序实际由三个线程组成, QThread 线程类的线程 Run()函数, 相关参数的设置, 数据包的投递都是在这个线程来完成的, 而实际的音频和视频的解码工作, 都是在各自的线程中完成的.

既然在各自的线程中完成, 那么就少不了一个问题, 就是如何做视频和音频的同步. 方法是, 在解码的时候每次都获取一下音频时钟(音频是连续, 准确的, 以此作为**主时钟**), 方法是每次解码音频的时候, 都计算音频时钟. 每一次解码视频的时候, 都获取一下当前视频对应显示时间 video_clock(pts), 如果发现视频显示时间大于音频的时间, 那么就延时(SDL_Delay(5)), 通过这种方式, 来让视音频同步.

简易解码流程:



添加 VideoState 结构体

在上节课的内容基础上进行调整.

添加 VideoState 结构体

在读取线程里面 (run()) 完成所有成员的赋值.

添加视频解码线程

然后, 添加视频解码线程 `int video_thread(void *arg).`

//视频解码线程函数

```

int video_thread(void *arg)
{
    VideoState *is = (VideoState *) arg;
    AVPacket pkt1, *packet = &pkt1;

    int ret, got_picture, numBytes;

    double video_pts = 0; //当前视频的 pts
    double audio_pts = 0; //音频 pts

    ///解码视频相关
    AVFrame *pFrame, *pFrameRGB;
    uint8_t *out_buffer_rgb; //解码后的 rgb 数据
    struct SwsContext *img_convert_ctx; //用于解码后的视频格式转换

    AVCodecContext *pCodecCtx = is->pCodecCtx; //视频解码器

    pFrame = av_frame_alloc();
    pFrameRGB = av_frame_alloc();

    ///这里我们改成了 将解码后的 YUV 数据转换成 RGB32
    img_convert_ctx = sws_getContext(pCodecCtx->width, pCodecCtx->height,
                                     pCodecCtx->pix_fmt, pCodecCtx->width, pCodecCtx->height,
                                     AV_PIX_FMT_RGB32, SWS_BICUBIC, NULL, NULL, NULL);

```

```

numBytes = avpicture_get_size(AV_PIX_FMT_RGB32,
pCodecCtx->width, pCodecCtx->height);

out_buffer_rgb = (uint8_t *) av_malloc(numBytes * sizeof(uint8_t));
avpicture_fill((AVPicture *) pFrameRGB, out_buffer_rgb, AV_PIX_FMT_RGB32,
pCodecCtx->width, pCodecCtx->height);

while(1)
{
    if (packet_queue_get(is->videoq, packet, 1) <= 0) break;//队列里面没有数据了
    读取完毕了
    while(1)
    {
        audio_pts = is->audio_clock;
        if (video_pts <= audio_pts) break;
        SDL_Delay(5);
    }
    ret = avcodec_decode_video2(pCodecCtx, pFrame, &got_picture, packet);
    if (ret < 0) {
        av_log(NULL, AV_LOG_ERROR, "Error decoding video frame\n");
        break;
    }
    //获取显示时间 pts
    video_pts = pFrame->pts = pFrame->best_effort_timestamp;
    video_pts *= 1000000 * av_q2d(is->video_st->time_base);
    video_pts = synchronize_video(is, pFrame, video_pts);//视频时钟补偿

    if (got_picture) {
        sws_scale(img_convert_ctx,
            (uint8_t const * const *) pFrame->data,
            pFrame->linesize, 0, pCodecCtx->height, pFrameRGB->data,
            pFrameRGB->linesize);

        //把这个 RGB 数据 用 QImage 加载
        QImage tmpImg((uchar
*)out_buffer_rgb, pCodecCtx->width, pCodecCtx->height, QImage::Format_RGB32);
        QImage image = tmpImg.copy(); //把图像复制一份 传递给界面显示
        is->m_player->SendGetOneImage(image); //调用激发信号的函数

    }
    av_free_packet(packet); //新版考虑使用 av_packet_unref() 函数来代替
}

```

```

    av_free(pFrame);
    av_free(pFrameRGB);
    av_free(out_buffer_rgb);

    return 0;
}

```

我们可以看出，上面的视频线程的内容，基本上都是 run() 函数 while(1) 中执行解码视频的代码。这里我们发送信号的函数，使用 SendGetOneFrame(image); 来完成的

//发送图片信号函数

```

void MediaPlayer::SendGetOneImage(QImage& img)
{
    emit SIG_GetOneImage(img); //发送信号
}

```

//时间补偿函数--视频延时

```

double synchronize_video(VideoState *is, AVFrame *src_frame, double pts) {

    double frame_delay; // 缓存帧和帧之间的延迟

    if (pts != 0) {
        /* if we have pts, set video clock to it */
        // 如果当前帧有 PTS 时间戳，那么使用它来更新视频时钟
        is->video_clock = pts;
    } else {
        /* if we aren't given a pts, set it to the clock */
        // 如果没有 PTS 时间戳，则采用视频时钟作为当前时间
        pts = is->video_clock;
    }

    /* update the video clock */
    // 计算当前帧和上一帧之间的时钟差
    frame_delay = av_q2d(is->video_st->codec->time_base);
    /* if we are repeating a frame, adjust clock accordingly */
    // 如果当前帧是重复帧，需要根据重复数调整帧之间的时间差
    frame_delay += src_frame->repeat_pict * (frame_delay * 0.5);
    // 更新视频时钟
    is->video_clock += frame_delay;
    // 返回当前帧的 PTS 时间戳
    return pts;
}

```

修改读取线程 run() 函数

然后我们修改 run() 函数里面的内容，绝大多数是不需要修改的结果如下：

```

#define    MAX_AUDIO_SIZE    (1024*16*25*10)//音频阈值
#define    MAX_VIDEO_SIZE    (1024*255*25*2)//视频阈值
//当队列里面的数据超过某个大小的时候 就暂停读取 防止一下子就把视频读完了，导致的空间分
配不足
void MediaPlayer::run()
{
    //开始解码获取图片
    //添加音频需要的变量
    int      audioStream = -1;//音频解码器需要的流的索引
    AVCodecContext *pAudioCodecCtx = NULL;//音频解码器信息指针
    AVCodec   *pAudioCodec = NULL; //音频解码器
    //SDL
    SDL_AudioSpec      wanted_spec; //SDL 音频设置
    SDL_AudioSpec      spec ; //SDL 音频设置
//视频
    AVCodecContext *pCodecCtx ; //视频的解码器信息指针
    AVCodec        *pCodec      ; //视频解码器
    AVPacket *packet;//读取解码前的包

//    AVFrame *pFrame, *pFrameRGB;// 用来存解码后的数据
//    int numBytes;//帧数据大小
//    uint8_t * out_buffer;//存储转化为 RGB 格式数据的缓冲区
//    static struct SwsContext *img_convert_ctx;//YUV 转 RGB 的结构

//1. 初始化 FFMPEG 调用了这个才能正常适用编码器和解码器 注册所用函数
av_register_all();

//SDL 初始化
if (SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO | SDL_INIT_TIMER))
{
    printf("Couldn't init SDL:%s\n", SDL_GetError());
    return;
}
memset ( &m_videoState , 0 , sizeof(VideoState) );

//2. 需要分配一个 AVFormatContext，FFMPEG 所有的操作都要通过这个 AVFormatContext 来
进行 可
//以理解为视频文件指针
AVFormatContext *pFormatCtx = avformat_alloc_context();

//中文兼容
std::string path = m_fileName.toStdString();
const char* file_path = path.c_str();

//3. 打开视频文件
if( avformat_open_input(&pFormatCtx, file_path, NULL, NULL) != 0 )

```

```

{
    qDebug() << "can't open file";
    return;
}
//3.1 获取视频文件信息
if (avformat_find_stream_info(pFormatCtx, NULL) < 0)
{
    qDebug() << "Could't find stream infomation.";
    return;
}
//4. 读取视频流

int videoStream = -1;

//查找音频视频流索引
if (find_stream_index(pFormatCtx, &videoStream, &audioStream) == -1)
{
    printf("Couldn't find stream index\n");
    return;
}

m_videoState.pFormatCtx = pFormatCtx;
m_videoState.videoStream = videoStream;
m_videoState.audioStream = audioStream;
m_videoState.m_player = this;
if (videoStream != -1)
{
    //5. 查找解码器
    pCodecCtx = pFormatCtx->streams[ videoStream ]->codec;
    pCodec = avcodec_find_decoder(pCodecCtx->codec_id);
    if (pCodec == NULL) {
        printf("Codec not found.");
        return;
    }
    //打开解码器
    if (avcodec_open2(pCodecCtx, pCodec, NULL) < 0) {
        printf("Could not open codec.");
        return;
    }

    //视频流
    m_videoState.video_st = pFormatCtx->streams[ videoStream ];
    m_videoState.pCodecCtx = pCodecCtx;
    //视频同步队列
    m_videoState.videoq = new PacketQueue;
    packet_queue_init( m_videoState.videoq);
}

```



```

        //创建视频线程
        m_videoState.video_tid = SDL_CreateThread( video_thread , "video_thread" ,
&m_videoState );

    }

    if( audioStream != -1 )
    {
        //5. 找到对应的音频解码器
        pAudioCodecCtx = pFormatCtx->streams[audioStream]->codec;
        pAudioCodec = avcodec_find_decoder(pAudioCodecCtx ->codec_id);
        if (!pAudioCodec)
        {
            printf( "Couldn't find decoder\n");
            return;
        } //打卡音频解码器
        avcodec_open2(pAudioCodecCtx, pAudioCodec, NULL);

        m_videoState.audio_st = pFormatCtx->streams[audioStream];
        m_videoState.pAudioCodecCtx = pAudioCodecCtx;
        //6. 设置音频信息， 用来打开音频设备。
        SDL_LockAudio();
        wanted_spec.freq      = pAudioCodecCtx->sample_rate;
        switch (pFormatCtx->streams[audioStream]->codec->sample_fmt)
        {
            case AV_SAMPLE_FMT_U8:
                wanted_spec.format = AUDIO_S8;
                break;
            case AV_SAMPLE_FMT_S16:
                wanted_spec.format = AUDIO_S16SYS;
                break;
            default:
                wanted_spec.format = AUDIO_S16SYS;
                break;
        };
        wanted_spec.channels   = pAudioCodecCtx->channels;           //通道数
        wanted_spec.silence    = 0;    //设置静音值
        wanted_spec.samples    = SDL_AUDIO_BUFFER_SIZE;             //读取第一帧后调整
        wanted_spec.callback   = audio_callback; //回调函数
        wanted_spec.userdata   = /*pAudioCodecCtx*/&m_videoState; //回调函数参数

        //7. 打开音频设备
        m_videoState.audioID = SDL_OpenAudioDevice( NULL , 0, &wanted_spec, &spec, 0);
        if( m_videoState.audioID < 0 ) //第二次打开 audio 会返回-1
        {
            printf( "Couldn't open Audio: %s\n", SDL_GetError());
            return;
        }
    }
}

```

```

}

//设置参数，供解码时候用，swr_alloc_set_opts 的 in 部分参数
switch (pFormatCtx->streams[audioStream]->codec->sample_fmt)
{
case AV_SAMPLE_FMT_U8:
    m_videoState.out_frame.format = AV_SAMPLE_FMT_U8;
    break;
case AV_SAMPLE_FMT_S16:
    m_videoState.out_frame.format = AV_SAMPLE_FMT_S16;
    break;
default:
    m_videoState.out_frame.format = AV_SAMPLE_FMT_S16;
    break;
};
m_videoState.out_frame.sample_rate = pAudioCodecCtx->sample_rate;
m_videoState.out_frame.channel_layout =
av_get_default_channel_layout(pAudioCodecCtx->channels);
m_videoState.out_frame.channels = pAudioCodecCtx->channels;

m_videoState.audioq = new PacketQueue;
//初始化队列
packet_queue_init(m_videoState.audioq);
m_videoState.audioFrame = av_frame_alloc();
SDL_UnlockAudio();
// SDL 播放声音 0 播放
SDL_PauseAudioDevice(m_videoState.audioID, 0);
}

```

packet = (AVPacket *) malloc(sizeof(AVPacket)); //分配一个 packet

//8. 循环读取视频帧，转换为 RGB 格式，抛出信号去控件显示

```

while(1)
{
    if( m_videoState.quit ) break;
    //这里做了个限制 当队列里面的数据超过某个大小的时候 就暂停读取 防止一下子就把
    视频读完了，导致的空间分配不足
    /* 这里 audioq.size 是指队列中的所有数据包带的音频数据的总量或者视频数据总量，并
    不是包的数量 */
    //这个值可以稍微写大一些
    if( m_videoState.audioStream != -1 && m_videoState.audioq->size >
MAX_AUDIO_SIZE ) {
        SDL_Delay(10);
        continue;
    }
    if ( m_videoState.videoStream != -1 && m_videoState.videoq->size >
MAX_VIDEO_SIZE) {

```

```
    SDL_Delay(10);  
    continue;  
}
```

//可以看出 av_read_frame 读取的是一帧视频，并存入一个 AVPacket 的结构中

```
if (av_read_frame(pFormatCtx, packet) < 0)  
{  
    if( m_videoState.quit ) break;  
    break; //这里认为视频读取完了  
}
```

//生成图片

```
if (packet->stream_index == m_videoState.videoStream)  
{  
    packet_queue_put(m_videoState.videoq, packet);  
}  
else if ( packet->stream_index == m_videoState.audioStream)  
{  
    packet_queue_put(m_videoState.audioq, packet);  
}  
else  
{  
    av_free_packet(packet);  
}
```

```
}
```

//9. 回收数据

```
while( m_videoState.videoStream != -1&& m_videoState.videoq->nb_packets != 0)  
{
```

```
    if( m_videoState .quit ) break;  
    SDL_Delay(100);  
}
```

```
SDL_Delay(100);
```

```
while( m_videoState.audioStream != -1 && m_videoState.audioq->nb_packets != 0)  
{
```

```
    if( m_videoState .quit ) break;  
    SDL_Delay(100);  
}
```

```
SDL_Delay(100);
```

//回收空间

```
if( audioStream != -1 )  
    avcodec_close(pAudioCodecCtx);
```

```
if( videoStream != -1 )  
    avcodec_close(pCodecCtx);
```

```

    avformat_close_input(&pFormatCtx);
    av_free( m_videoState.audioFrame );
    m_videoState.audioFrame = NULL;
}

```

需要修改的地方，都标注出来了。

音频回调函数

//13. 回调函数中将从队列中取数据，解码后填充到播放缓冲区.

```

void audio_callback(void *userdata, Uint8 *stream, int len)
{
    // AVCodecContext *pcodec_ctx = (AVCodecContext *) userdata;
    VideoState * is = (VideoState *) userdata;

    int len1, audio_data_size;

    // static uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
    // static unsigned int audio_buf_size = 0;
    // static unsigned int audio_buf_index = 0;

    /* len 是由 SDL 传入的 SDL 缓冲区的大小，如果这个缓冲未滿，我们就一直往里填充数据 */

    /* audio_buf_index 和 audio_buf_size 标示我们自己用来放置解码出来的数据的缓冲区，*/
    /* 这些数据待 copy 到 SDL 缓冲区， 当 audio_buf_index >= audio_buf_size 的时候意味着我*/
    /* 们的缓冲为空，没有数据可供 copy，这时候需要调用 audio_decode_frame 来解码出更*/
    /* 多的帧数据 */
    while (len > 0)
    {
        if (is->audio_buf_index >= is->audio_buf_size) {
            audio_data_size = audio_decode_frame( is ,
            is->audio_buf, sizeof(is->audio_buf));
            /* audio_data_size < 0 标示没能解码出数据，我们默认播放静音 */
            if (audio_data_size < 0) {
                /* silence */
                is->audio_buf_size = 1024;
                /* 清零，静音 */
                memset(is->audio_buf, 0, is->audio_buf_size);
            } else {
                is->audio_buf_size = audio_data_size;
            }
            is->audio_buf_index = 0;
        }
        /* 查看 stream 可用空间，决定一次 copy 多少数据，剩下的下次继续 copy */
        len1 = is->audio_buf_size - is->audio_buf_index;
    }
}

```

```

    if (len1 > len) {
        len1 = len;
    }

    memset( stream , 0 , len1);
    //混音函数 sdl 2.0 版本使用该函数 替换 SDL_MixAudio
    SDL_MixAudioFormat(stream, (uint8_t *) is->audio_buf + is->audio_buf_index,
                        AUDIO_S16SYS, len1, 100);

    len -= len1;
    stream += len1;
    is->audio_buf_index += len1;
}
}

```

音频解码函数.

//对于音频来说, 一个 packet 里面, 可能含有多帧(frame)数据。

//解码音频帧函数

```

int audio_decode_frame(VideoState *is, uint8_t *audio_buf, int buf_size)
{
    AVPacket pkt;
    uint8_t *audio_pkt_data = NULL;
    int audio_pkt_size = 0;
    int len1, data_size;
    int sampleSize = 0;
    AVCodecContext *aCodecCtx = is->pAudioCodecCtx;
    AVFrame *audioFrame = is->audioFrame/*av_frame_alloc()*/;
    PacketQueue *audioq = is->audioq;
    AVFrame wanted_frame = is->out_frame;
    if( !aCodecCtx || !audioFrame || !audioq) return -1;
    /*static*/ struct SwrContext *swr_ctx = NULL;
    int convert_len;
    int n = 0;
    for(;;)
    {
        if( is->quit ) return -1;
        if( !audioq ) return -1;
        if(packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
        {
            return -1;
        }
        audio_pkt_data = pkt.data;
        audio_pkt_size = pkt.size;
        while(audio_pkt_size > 0)
        {
            if( is->quit ) return -1;
            int got_picture;
            memset(audioFrame, 0, sizeof(AVFrame));
            int ret =avcodec_decode_audio4( aCodecCtx, audioFrame, &got_picture, &pkt);

```

```

if( ret < 0 ) {
    printf("Error in decoding audio frame.\n");
    exit(0);
}
//一帧一个声道读取数据字节数是 nb_samples , channels 为声道数 2 表示 16 位
2 个字节

//data_size = audioFrame->nb_samples * wanted_frame.channels * 2;
switch( is->out_frame.format )
{
case AV_SAMPLE_FMT_U8:
    data_size = audioFrame->nb_samples * is->out_frame.channels * 1;
    break;
case AV_SAMPLE_FMT_S16:
    data_size = audioFrame->nb_samples * is->out_frame.channels * 2;
    break;
default:
    data_size = audioFrame->nb_samples * is->out_frame.channels * 2;
    break;
}
//计算音频时钟
if( pkt.pts != AV_NOPTS_VALUE)
{
    is->audio_clock = pkt.pts *av_q2d( is->audio_st->time_base )*1000000 ;
//取音频时钟
}else
{
    is->audio_clock = (*(uint64_t *)
audioFrame->opaque)*av_q2d( is->audio_st->time_base )*1000000 ;
}

if( got_picture )
{
    swr_ctx = swr_alloc_set_opts(NULL, wanted_frame.channel_layout,
(AVSampleFormat)wanted_frame.format,wanted_frame.sample_rate,
audioFrame->channel_layout, (AVSampleFormat)audioFrame->format,
audioFrame->sample_rate, 0, NULL);

//初始化
if (swr_ctx == NULL || swr_init(swr_ctx) < 0)
{
    printf("swr_init error\n");
    break;
}
convert_len = swr_convert(swr_ctx, &audio_buf,
AVCODEC_MAX_AUDIO_FRAME_SIZE,

```

```

        (const uint8_t **)audioFrame->data,
        audioFrame->nb_samples);

    swr_free( &swr_ctx);
}
audio_pkt_size -= ret;
if (audioFrame->nb_samples <= 0)
{
    continue;
}
av_free_packet(&pkt); //新版考虑使用 av_packet_unref() 函数来代替
return data_size ;
}
av_free_packet(&pkt); //新版考虑使用 av_packet_unref() 函数来代替
}
}

//查找数据流函数
int find_stream_index(AVFormatContext *pformat_ctx, int *video_stream, int
*audio_stream)
{
    assert(video_stream != NULL || audio_stream != NULL);

    int i = 0;
    int audio_index = -1;
    int video_index = -1;

    for (i = 0; i < pformat_ctx->nb_streams; i++)
    {
        if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_VIDEO)
        {
            video_index = i;
        }
        if (pformat_ctx->streams[i]->codec->codec_type == AVMEDIA_TYPE_AUDIO)
        {
            audio_index = i;
        }
    }

    //注意以下两个判断有可能返回-1.
    if (video_stream == NULL)
    {
        *audio_stream = audio_index;
        return *audio_stream;
    }

    if (audio_stream == NULL)
    {
        *video_stream = video_index;
    }
}

```

```
        return *video_stream;
    }

    *video_stream = video_index;
    *audio_stream = audio_index;

    return 0;
}
```

注意事项

注意:重新构建.